Challenge: Make A Camera See Fast Enough To Act On

The Tenstorrent team welcomes you, and thanks you for choosing this track. We hope you enjoy the work, and that you come away knowing something about Tenstorrent hardware and the open software ecosystem around it.

1 Challenge and scope

A machine that moves must recognize what is in front of it before it can act. Doing that takes time, and while it takes time the machine is moving blind. Your job is to cut that time on a Tenstorrent card without losing the ability to see.

Work to one machine, the same for every team: a vehicle at 50 km/h. That is 13.9 meters a second, so it covers 13.9 millimetres for every millisecond of delay. At the current 69.122 milliseconds a frame it travels 96 centimetres before it knows what it saw. At 15 milliseconds it travels 21 centimetres. Under 15 milliseconds is reachable on this card with no measurable loss of accuracy.

2 Current measurements on Tenstorrent hardware

One photograph through the full loop, timed on the card you will be given. Mean of 300 frames at 320x320 on a Wormhole n150, after discarding 20 warm-up frames.

These are measurements, not estimates. The run that produced them wrote its output to ~/challenge/data on your machine, together with a record of the board, the firmware and the software version used. Run bash ~/challenge/bench/run_bench.sh and you will get the same table for your own card. Expect it to differ by a few percent: two physically identical cards measured 69.122 and 66.0 milliseconds for the same work.

stage	ms	what actually happens
unpack the photo	2.135	turn the compressed JPEG back into pixels
resize and convert	0.827	shrink to 320x320, put the colours in the order the model wants
hand it to the card	2.903	copy the pixels into the card's own memory
run the detector	61.062	YOLOv4, on the card
work out the boxes	2.195	pick the likeliest object for each box, drop the overlaps
whole frame, measured	**69.122**	**one stopwatch across all five, not a sum of the rows**

3 What we provide

A working object detector, already installed and running on your machine: YOLOv4 in TTNN, which recognises 80 everyday categories and is the thing you will be making faster. It is written out as seven stages in separate files, so you can change one piece at a time, and it comes with a plain PyTorch version to compare against and tests that tell you which stage you broke.

A machine in the cloud with a Tenstorrent card in it, reached through a browser. Nothing to install, no accounts to create, alive for the whole event.

Under ~/challenge: measurement tools that time the loop stage by stage, a live camera demonstration, the raw data behind every number above, the scoring harness with a template to fill in, and a two-photograph sample so you can prove your submission is the right shape on day one. The README there covers everything not in this document, including the confidence and overlap thresholds and two traps that have already cost us a day each.

Two hundred practice photographs with their answers recorded, in ~/challenge/practice. They are not the photographs you are scored on and do not overlap them. Point the scoring tool at them and every change you make gives you a speed and an accuracy together, which is how you find out whether an optimisation cost you anything before it is too late to act on it.

4 Rules

You may change anything, including the model. A newer detector is allowed, but getting one to run well on this hardware is a day's work, so if you try, have something working end to end by hour 24.
You are scored on two numbers over the same 800 photographs: how accurate your detections are, measured against recorded answers, and how many frames a second you manage. Neither is worth anything alone. A fast version that misses half the objects scores badly, and so does an accurate version no quicker than the one you started with.
The tool that scores you is in ~/challenge/harness and you have it from the first hour, so build against it from the start.

5 What you hand in

Your submission file. One file, which you make by copying ~/challenge/harness/submission_template.py and filling in the four places it marks. It holds the one configuration you are claiming: which model, what resolution, which thresholds, and everything you did to make it fast. This is the file that runs against the 800 photographs at hour 30.
A one-page log of what you changed, in order, with what each change was worth. Measure both halves: speed with bench/run_bench.sh, and accuracy by running the scoring tool against the practice photographs. Each such run is one point, and several of them are your trade-off curve. harness/plot_curve.py draws it. Say which numbers you measured and which you estimated.
Your settings stated: the resolution you ran at, which model you used, and the confidence and overlap thresholds you chose. All three are yours to pick and all three move the numbers, so they have to be written down. The README explains what they do.
Your own machine's starting measurement, taken before you changed anything.

6 How it is judged

At hour 30 the 800 photographs unlock and your submission runs against them. Accuracy is measured once, because it does not vary between runs. Speed is measured three times and the fastest reading is the one that counts, because other work on the machine can only add time to a run and never take it away, so the fastest of several readings is the honest figure for your own code. Those two numbers are your position on the chart. Every team is plotted together with the detector as supplied, so who went faster without losing sight is visible at a glance. We run it ourselves, on your machine, with you watching, so one piece of code produces every team's result.

7 Scoring criteria

Every team is scored on the same five criteria. Each criterion is scored from 0 to 5. The challenge-specific criterion counts double (2x), so the maximum is 30 points.

Criterion	Weight	What judges look at
Technical Feasibility	1x	Does the solution work, and is the approach technically sound? Judges look for a working prototype or a credible technical path, and for evidence that is measured, reproducible and honest about its limits.
Innovation	1x	How new is the idea or approach compared with the current state of the art, and how creative is the way it tackles the problem?
Market Potential	1x	Is there a real need, a clear customer or user, and room to grow? Judges look at how large and how reachable the opportunity is.
Business Viability	1x	Could this become a sustainable product or business? Judges look at the business model, the cost and effort of adoption, the route to market and what it would take to get there.
Challenge-specific: Speed and accuracy result	**2x**	Frames per second and detection accuracy over the same 800 photographs at hour 30, plotted against the detector as supplied.

Total: 6 weighted units (1 + 1 + 1 + 1 + 2), maximum 30 points.

7.1 Challenge-specific criterion: Speed and accuracy result (2x)

• Accuracy: how accurate your detections are against the recorded answers, measured once over the 800 photographs.
• Speed: frames per second, measured three times, with the fastest reading counting.
• Neither number is worth anything alone. Every team is plotted together with the detector as supplied, so who went faster without losing sight is visible at a glance.

All the best. Three Tenstorrent engineers are on site for the whole event. Ask early rather than losing a day to something one of us has already hit.